MoA Prediction — Hyperparameter Tuning Pipeline

This notebook combines the two-stage tuning strategy with feature engineering and pre-calculated stratified folds. **Time limits and deadline prunings have been completely removed** to ensure that all candidate models and default configurations are fully evaluated without skipping any step due to time constraints.

Key Features:

- **Advanced Feature Engineering**: Row-wise statistics (`g-` and `c-`), QuantileTransformer, PCA for ResNet, and raw features for FT-Transformer and CatBoost.
- **Grouping and Folds**: `MultilabelStratifiedKFold` with 5 folds on the entire dataset.
- **Two-Stage Tuning Strategy (Unlimited Time)**:
  1. **Stage 1 (Search)**: Fixed-trial Optuna search on Fold 0 (`SEARCH_FOLD`) with early-stopping pruner on internal validation loss (10% inner split).
  2. **Stage 2 (Confirmation)**: Unconditional re-evaluation of top K candidates and **default** configuration on Fold 1 (`CONFIRM_FOLD`). No candidate is skipped due to insufficient time.
- **Decision Rule**: A tuned model is adopted in the final JSON file only if it beats the default configuration on **both** folds with a minimum average gain ($\geq \mathrm{MIN_GAIN}$).
- **GPU Stability on Kaggle (2x T4)**: Sequential execution and direct VRAM management to prevent CUDA concurrency errors (`DeadKernelError`).

Workflow

1. Install
2. Imports and configuration
3. Data loading and fold pre-computation
4. Models
5. Training and evaluation functions
6. Search spaces and evaluators
7. Stage 2 and decision rule
8. Pipeline

9. Execution

In [1]:
```
# Step 1 - Install Dependencies
!pip install -q iterative-stratification optuna catboost
```

In [2]:
```
# Step 2 - Imports and Global Configuration
import os
import gc
import json
import glob
import time
import math
import traceback
import warnings

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import optuna
from sklearn.decomposition import PCA
from sklearn.preprocessing import QuantileTransformer
from sklearn.feature_selection import VarianceThreshold
from sklearn.exceptions import DataConversionWarning
from iterstrat.ml_stratifiers import MultilabelStratifiedKFold, MultilabelStratifiedShuffleSplit
from catboost import CatBoostClassifier

warnings.filterwarnings("ignore", category=DataConversionWarning)
warnings.filterwarnings("ignore")
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Constants and Parameters
SMOKE_TEST = False              # If True, runs a fast test (~5 min) on a reduced subset
DATA_DIR = "/kaggle/input/competitions/lish-moa/"
OUTPUT_DIR = "/kaggle/working"
SEED = 42
RANDOM_STATE = 42
CTL_LABEL = "ctl_vehicle"
CLIP_EPS = 1e-15

# Folds & Two-stage evaluation
N_FOLDS = 5
SEARCH_FOLD = 0                 # Fold for Stage 1 (search)
CONFIRM_FOLD = 1                # Fold for Stage 2 (confirmation)
```

```
# Preprocessing & Feature Engineering
N_QUANTILES = 100
PCA_G_COMPONENTS = 50
PCA_C_COMPONENTS = 15
VARIANCE_THRESHOLD = 0.0

# Training Recipe
INNER_VAL_FRACTION = 0.1
LABEL_SMOOTHING = 0.001
PLATEAU_FACTOR, PLATEAU_PATIENCE = 0.1, 3
RESNET_MAX_EPOCHS, RESNET_PATIENCE = 35, 8
FT_MAX_EPOCHS, FT_PATIENCE = 35, 8
EVAL_BATCH_SIZE = {"resnet": 1024, "ft": 128}
CATBOOST_ITERATIONS = 250

# Tuning Parameters (Number of search trials replacing time limits)
N_TRIALS_SEARCH = {"resnet": 25, "catboost": 15, "ft": 15}
FT_SEARCH_ARCHITECTURE = False
K_CONFIRM = 3
MIN_GAIN = 5e-5
MIN_TRIALS_FOR_A_SEARCH = 8

if SMOKE_TEST:
    N_ROWS_SMOKE = 4000
    N_TRIALS_SEARCH = {"resnet": 3, "catboost": 2, "ft": 2}
    RESNET_MAX_EPOCHS, FT_MAX_EPOCHS = 3, 2
    RESNET_PATIENCE, FT_PATIENCE = 2, 1
    CATBOOST_ITERATIONS = 20
    K_CONFIRM = 1
    MIN_TRIALS_FOR_A_SEARCH = 2

N_GPUS = torch.cuda.device_count()
DEVICE = torch.device("cuda:0" if N_GPUS > 0 else "cpu")
os.makedirs(OUTPUT_DIR, exist_ok=True)
print(f"Device in use: {DEVICE} | Available GPUs: {N_GPUS} | Smoke Test: {SMOKE_TEST}")
```

```
Device in use: cuda:0 | Available GPUs: 2 | Smoke Test: False
```

In [3]:

```
# Step 3 - Data Loading and Pre-computing Fold Transformations
def load_and_preprocess_data(data_dir=DATA_DIR):
    if not os.path.exists(os.path.join(data_dir, "train_features.csv")):
        matches = glob.glob(f"/kaggle/input/**/train_features.csv", recursive=True)
        if len(matches) > 0:
            data_dir = os.path.dirname(matches[0]) + "/"

    train_features = pd.read_csv(f"{data_dir}train_features.csv")
    train_targets = pd.read_csv(f"{data_dir}train_targets_scored.csv")
```

```
    # Remove control samples (ctl_vehicle)
    non_control_mask = train_features["cp_type"] != CTL_LABEL
    train_features = train_features[non_control_mask].reset_index(drop=True)
    train_targets = train_targets[non_control_mask].reset_index(drop=True)

    feature_cols = [c for c in train_features.columns if c not in ["sig_id", "cp_type"]]
    target_cols = [c for c in train_targets.columns if c != "sig_id"]
    g_cols = [c for c in feature_cols if c.startswith("g-")]
    c_cols = [c for c in feature_cols if c.startswith("c-")]

    # Row-wise statistics for g- and c- features
    df_out = train_features[[c for c in train_features.columns if c != "cp_type"]].copy()
    df_out["g_sum"] = df_out[g_cols].sum(axis=1)
    df_out["g_mean"] = df_out[g_cols].mean(axis=1)
    df_out["g_std"] = df_out[g_cols].std(axis=1)
    df_out["g_kurtosis"] = df_out[g_cols].kurtosis(axis=1)
    df_out["g_skew"] = df_out[g_cols].skew(axis=1)

    df_out["c_sum"] = df_out[c_cols].sum(axis=1)
    df_out["c_mean"] = df_out[c_cols].mean(axis=1)
    df_out["c_std"] = df_out[c_cols].std(axis=1)
    df_out["c_kurtosis"] = df_out[c_cols].kurtosis(axis=1)
    df_out["c_skew"] = df_out[c_cols].skew(axis=1)

    dummy_cols = ["cp_time", "cp_dose"]
    df_out = pd.get_dummies(df_out, columns=dummy_cols, dtype=np.float64)

    full_feature_cols = [c for c in df_out.columns if c != "sig_id"]
    dummy_feature_cols = [c for c in full_feature_cols if any(c.startswith(d) for d in dummy_cols)]

    X = df_out[full_feature_cols].values.astype(np.float64)
    Y = train_targets[target_cols].values

    if SMOKE_TEST:
        keep = np.sort(np.random.default_rng(SEED).choice(len(X), size=min(N_ROWS_SMOKE, len(X)), replace=False))
        X, Y = X[keep], Y[keep]

    return X, Y, full_feature_cols, g_cols, c_cols, dummy_feature_cols


def prepare_cached_folds(X, Y, feature_cols, g_cols, c_cols, dummy_cols):
    """Pre-computes PCA and Raw transformations on each stratified fold (MultilabelStratifiedKFold)"""
    print(f"Pre-computing {N_FOLDS} stratified folds...")
    mskf = MultilabelStratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_STATE)
    cached_folds = {}
    as32 = lambda arr: np.ascontiguousarray(arr, dtype=np.float32)

    for fold_id, (train_idx, val_idx) in enumerate(mskf.split(X, Y)):
```

```
        if fold_id not in (SEARCH_FOLD, CONFIRM_FOLD):
            continue
        X_train, X_val = X[train_idx], X[val_idx]
        Y_train, Y_val = Y[train_idx], Y[val_idx]

        # Variance Threshold
        selector = VarianceThreshold(threshold=VARIANCE_THRESHOLD)
        X_train_vt = selector.fit_transform(X_train)
        X_val_vt = selector.transform(X_val)
        support = selector.get_support()
        selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

        dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
        cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]
        cont_cols = [selected_cols[i] for i in cont_idx]
        g_idx = [i for i, c in enumerate(cont_cols) if c in g_cols]
        c_idx = [i for i, c in enumerate(cont_cols) if c in c_cols]

        # QuantileTransformer
        qt = QuantileTransformer(n_quantiles=N_QUANTILES, output_distribution="normal", random_state=RANDOM_STATE)
        tf_train = qt.fit_transform(X_train_vt[:, cont_idx])
        tf_val = qt.transform(X_val_vt[:, cont_idx])

        # PCA
        pca_g = PCA(n_components=PCA_G_COMPONENTS, random_state=RANDOM_STATE)
        g_train = pca_g.fit_transform(tf_train[:, g_idx])
        g_val = pca_g.transform(tf_val[:, g_idx])

        pca_c = PCA(n_components=PCA_C_COMPONENTS, random_state=RANDOM_STATE)
        c_train = pca_c.fit_transform(tf_train[:, c_idx])
        c_val = pca_c.transform(tf_val[:, c_idx])

        dummy_train, dummy_val = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]

        # Inner Split (10%) for Early Stopping and Scheduler during training
        msss = MultilabelStratifiedShuffleSplit(n_splits=1, test_size=INNER_VAL_FRACTION, random_state=SEED)
        fit_idx, inner_idx = next(msss.split(X_train, Y_train))

        cached_folds[fold_id] = {
            "fold": fold_id,
            "nn_train": as32(np.hstack([tf_train, g_train, c_train, dummy_train])),
            "nn_val": as32(np.hstack([tf_val, g_val, c_val, dummy_val])),
            "raw_train": as32(np.hstack([tf_train, dummy_train])),
            "raw_val": as32(np.hstack([tf_val, dummy_val])),
            "Y_train": Y_train, "Y_val": Y_val,
            "fit_idx": fit_idx, "inner_idx": inner_idx
        }
    return cached_folds
```

In [4]:

```python
# Step 4 - Models (TabularResNet, FTTransformer)
class TabularResNet(nn.Module):
    name = "resnet"
    def __init__(self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2):
        super().__init__()
        self.in_proj = nn.Sequential(
            nn.BatchNorm1d(input_dim),
            nn.Dropout(dropout_rate),
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
        )
        self.block1 = self._make_block(hidden_dim, dropout_rate)
        self.block2 = self._make_block(hidden_dim, dropout_rate)
        self.out_proj = nn.Sequential(
            nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
        )

    @staticmethod
    def _make_block(hidden_dim, dropout_rate):
        return nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
        )

    def forward(self, x):
        x = self.in_proj(x)
        x = F.silu(x + self.block1(x))
        x = F.silu(x + self.block2(x))
        return self.out_proj(x)


class FeatureTokenizer(nn.Module):
    def __init__(self, num_features, d_token):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_features, d_token))
        self.bias = nn.Parameter(torch.empty(num_features, d_token))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.zeros_(self.bias)

    def forward(self, x):
        return x.unsqueeze(-1) * self.weight + self.bias
```

```
class FTTransformer(nn.Module):
    name = "ft_transformer"
    def __init__(self, input_dim, output_dim=206, d_token=64, n_blocks=2,
                 attention_heads=4, dropout=0.2):
        super().__init__()
        self.tokenizer = FeatureTokenizer(input_dim, d_token)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
        nn.init.normal_(self.cls_token, std=0.02)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token, nhead=attention_heads, dim_feedforward=d_token * 4,
            dropout=dropout, activation="gelu", batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_blocks, enable_nested_tensor=False)
        self.layer_norm = nn.LayerNorm(d_token)
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(d_token, output_dim))

    def forward(self, x):
        tokens = self.tokenizer(x)
        cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat((cls_tokens, tokens), dim=1)
        x = self.transformer(x)
        cls_out = self.layer_norm(x[:, 0, :])
        return self.head(cls_out)
```

In [5]:

```
# Step 5 - Training and Evaluation Functions
def log_loss_float64(y_true, probabilities, eps=CLIP_EPS):
    p = np.clip(np.asarray(probabilities, dtype=np.float64), eps, 1 - eps)
    y = np.asarray(y_true, dtype=np.float64)
    return float(np.mean(-(y * np.log(p) + (1 - y) * np.log(1 - p))))


@torch.no_grad()
def predict_probabilities(model, X, batch_size, use_amp):
    model.eval()
    chunks = []
    for start in range(0, X.shape[0], batch_size):
        with torch.autocast(device_type=X.device.type, dtype=torch.float16, enabled=use_amp):
            logits = model(X[start:start + batch_size])
        chunks.append(torch.sigmoid(logits.float()).cpu().numpy())
    return np.vstack(chunks).astype(np.float64)


def make_gpu_data(train_array, val_array, fold, device):
    X_train = torch.as_tensor(train_array, dtype=torch.float32, device=device)
    y_train = torch.as_tensor(fold["Y_train"], dtype=torch.float32, device=device)
    fit_idx = torch.as_tensor(fold["fit_idx"], dtype=torch.long, device=device)
```

```
    inner_idx = torch.as_tensor(fold["inner_idx"], dtype=torch.long, device=device)
    return {
        "X_fit": X_train[fit_idx], "y_fit": y_train[fit_idx],
        "X_inner": X_train[inner_idx], "y_inner": fold["Y_train"][fold["inner_idx"]],
        "X_val": torch.as_tensor(val_array, dtype=torch.float32, device=device), "y_val": fold["Y_val"]
    }


def train_and_score(model, data, *, lr, weight_decay, batch_size, max_epochs, patience, eval_batch_size,
                    device, trial=None, seed=SEED):
    use_amp = device.type == "cuda"
    model = model.to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=PLATEAU_FACTOR, patience=PLATEAU_PATIENCE)
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

    X_fit, y_fit = data["X_fit"], data["y_fit"]
    y_smoothed = y_fit * (1 - LABEL_SMOOTHING) + 0.5 * LABEL_SMOOTHING
    n_rows = X_fit.shape[0]
    steps_per_epoch = max(1, n_rows // batch_size)
    generator = torch.Generator(device=device)
    generator.manual_seed(seed)

    best_inner_loss, best_state, epochs_without_improvement, epochs_run = float("inf"), None, 0, 0
    for epoch in range(max_epochs):
        model.train()
        order = torch.randperm(n_rows, device=device, generator=generator)
        for step in range(steps_per_epoch):
            batch = order[step * batch_size:(step + 1) * batch_size]
            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                logits = model(X_fit[batch])
            loss = criterion(logits.float(), y_smoothed[batch])
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

        inner_loss = log_loss_float64(data["y_inner"], predict_probabilities(model, data["X_inner"], eval_batch_size, use_amp))
        if not np.isfinite(inner_loss):
            raise optuna.TrialPruned()
        scheduler.step(inner_loss)
        epochs_run += 1
        if inner_loss < best_inner_loss:
            best_inner_loss = inner_loss
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
            epochs_without_improvement = 0
        else:
```

```
            epochs_without_improvement += 1
        if trial is not None:
            trial.report(inner_loss, epoch)
            if trial.should_prune():
                raise optuna.TrialPruned()
        if epochs_without_improvement >= patience:
            break

    if best_state is None:
        raise optuna.TrialPruned()
    model.load_state_dict(best_state)
    val_loss = log_loss_float64(data["y_val"], predict_probabilities(model, data["X_val"], eval_batch_size, use_amp))
    return {"val_loss": val_loss, "best_inner_loss": float(best_inner_loss), "epochs": epochs_run}


def run_guarded(function, *args, **kwargs):
    try:
        return function(*args, **kwargs)
    except torch.cuda.OutOfMemoryError:
        print("  CUDA out of memory: trial pruned")
        raise optuna.TrialPruned()
    finally:
        gc.collect()
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
```

In [6]:

```
# Step 6 - Search Spaces and Model Evaluators
RESNET_DEFAULT = {"lr": 1e-3, "dropout": 0.2, "weight_decay": 1e-5, "hidden_dim": 512, "batch_size": 256}
FT_DEFAULT = {"lr": 1e-3, "dropout": 0.2, "weight_decay": 1e-5, "d_token": 64, "n_blocks": 2, "attention_heads": 4, "batch_size": 64}
CATBOOST_DEFAULT = {"depth": 5, "l2_leaf_reg": 3.0, "learning_rate": 0.08, "subsample": None, "iterations": CATBOOST_ITERATIONS}
DEFAULT_CONFIG = {"resnet": RESNET_DEFAULT, "ft": FT_DEFAULT, "catboost": CATBOOST_DEFAULT}

DEFAULT_SUGGESTED = {
    "resnet": dict(RESNET_DEFAULT),
    "ft": {k: FT_DEFAULT[k] for k in ("lr", "dropout", "weight_decay", "batch_size")
           + (("d_token", "n_blocks", "attention_heads") if FT_SEARCH_ARCHITECTURE else ())},
    "catboost": {"depth": 5, "l2_leaf_reg": 3.0, "learning_rate": 0.08, "bootstrap": "default"},
}

def suggest_resnet(trial):
    return {
        "lr": trial.suggest_float("lr", 3e-4, 3e-3, log=True),
        "dropout": trial.suggest_float("dropout", 0.1, 0.45),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True),
        "hidden_dim": trial.suggest_categorical("hidden_dim", [256, 512, 768]),
        "batch_size": trial.suggest_categorical("batch_size", [128, 256, 512])
    }
```

```python
def suggest_ft(trial):
    params = {
        "lr": trial.suggest_float("lr", 3e-4, 3e-3, log=True),
        "dropout": trial.suggest_float("dropout", 0.05, 0.35),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True),
        "batch_size": trial.suggest_categorical("batch_size", [64, 128])
    }
    if FT_SEARCH_ARCHITECTURE:
        params.update(
            d_token=trial.suggest_categorical("d_token", [32, 64]),
            n_blocks=trial.suggest_categorical("n_blocks", [1, 2]),
            attention_heads=trial.suggest_categorical("attention_heads", [2, 4])
        )
    return params

def suggest_catboost(trial):
    params = {
        "depth": trial.suggest_int("depth", 4, 6),
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1.0, 20.0, log=True),
        "learning_rate": trial.suggest_float("learning_rate", 0.04, 0.25, log=True),
        "bootstrap": trial.suggest_categorical("bootstrap", ["default", "Bernoulli"])
    }
    if params["bootstrap"] == "Bernoulli":
        params["subsample"] = trial.suggest_float("subsample", 0.6, 0.95)
    return params

SUGGEST = {"resnet": suggest_resnet, "ft": suggest_ft, "catboost": suggest_catboost}

def build_config(kind, params):
    if kind in ("resnet", "ft"):
        base = DEFAULT_CONFIG[kind]
        return {**base, **{key: value for key, value in params.items() if key in base}}
    config = dict(CATBOOST_DEFAULT)
    for key in ("depth", "l2_leaf_reg", "learning_rate"):
        if key in params:
            config[key] = params[key]
    config["subsample"] = params.get("subsample") if params.get("bootstrap") == "Bernoulli" else None
    return config

def evaluate_resnet(config, fold, device, trial=None):
    data = make_gpu_data(fold["nn_train"], fold["nn_val"], fold, device)
    n_inputs, n_outputs = data["X_fit"].shape[1], fold["Y_train"].shape[1]
    torch.manual_seed(SEED)
    model = TabularResNet(n_inputs, n_outputs, hidden_dim=config["hidden_dim"], dropout_rate=config["dropout"])
    return train_and_score(model, data, lr=config["lr"], weight_decay=config["weight_decay"],
                           batch_size=config["batch_size"], max_epochs=RESNET_MAX_EPOCHS, patience=RESNET_PATIENCE,
                           eval_batch_size=EVAL_BATCH_SIZE["resnet"], device=device, trial=trial)
```

```
def evaluate_ft(config, fold, device, trial=None):
    data = make_gpu_data(fold["raw_train"], fold["raw_val"], fold, device)
    n_inputs, n_outputs = data["X_fit"].shape[1], fold["Y_train"].shape[1]
    torch.manual_seed(SEED)
    model = FTTransformer(n_inputs, n_outputs, d_token=config["d_token"], n_blocks=config["n_blocks"],
                          attention_heads=config["attention_heads"], dropout=config["dropout"])
    return train_and_score(model, data, lr=config["lr"], weight_decay=config["weight_decay"],
                           batch_size=config["batch_size"], max_epochs=FT_MAX_EPOCHS, patience=FT_PATIENCE,
                           eval_batch_size=EVAL_BATCH_SIZE["ft"], device=device, trial=trial)

def evaluate_catboost(config, fold, device, trial=None):
    Y_train, Y_val = fold["Y_train"], fold["Y_val"]
    is_constant = Y_train.min(axis=0) == Y_train.max(axis=0)
    active, constant = np.where(~is_constant)[0], np.where(is_constant)[0]
    prior = (Y_train.sum(axis=0) + 0.5) / (Y_train.shape[0] + 1.0)
    params = dict(loss_function="MultiLogloss", iterations=config["iterations"], depth=config["depth"],
                  learning_rate=config["learning_rate"], l2_leaf_reg=config["l2_leaf_reg"],
                  random_seed=SEED, verbose=False, allow_writing_files=False)
    if config["subsample"] is not None:
        params.update(bootstrap_type="Bernoulli", subsample=config["subsample"])
    if N_GPUS > 0:
        params.update(task_type="GPU", devices="0")
    model = CatBoostClassifier(**params)
    model.fit(fold["raw_train"], Y_train[:, active])
    predictions = np.zeros(Y_val.shape, dtype=np.float64)
    predictions[:, active] = model.predict_proba(fold["raw_val"])
    if constant.size:
        predictions[:, constant] = prior[constant]
    return {"val_loss": log_loss_float64(Y_val, predictions), "best_inner_loss": None, "epochs": None}

EVALUATE = {"resnet": evaluate_resnet, "ft": evaluate_ft, "catboost": evaluate_catboost}

def make_objective(kind, fold, device, durations):
    def objective(trial):
        params = SUGGEST[kind](trial)
        config = build_config(kind, params)
        start = time.time()
        result = run_guarded(EVALUATE[kind], config, fold, device, trial)
        seconds = time.time() - start
        durations.append(seconds)
        trial.set_user_attr("seconds", round(seconds, 1))
        if result.get("epochs") is not None:
            trial.set_user_attr("epochs", result["epochs"])
            trial.set_user_attr("best_inner_loss", result["best_inner_loss"])
        return result["val_loss"]
    return objective
```

In [7]:

```python
# Step 7 - Stage 2 and Decision Rule (Unrestricted Evaluation)
LABEL = {"resnet": "ResNet", "ft": "FTTransformer", "catboost": "CatBoost"}
N_STARTUP = {"resnet": 10, "ft": 4, "catboost": 4}
PRUNER_FACTORY = {
    "resnet": lambda: optuna.pruners.MedianPruner(n_startup_trials=8, n_warmup_steps=6),
    "ft": lambda: optuna.pruners.MedianPruner(n_startup_trials=3, n_warmup_steps=6),
    "catboost": lambda: optuna.pruners.NopPruner()
}
COMPLETE = optuna.trial.TrialState.COMPLETE

def to_jsonable(obj):
    if isinstance(obj, dict):
        return {str(key): to_jsonable(value) for key, value in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [to_jsonable(value) for value in obj]
    if isinstance(obj, np.generic):
        return obj.item()
    return obj

def make_callback(kind):
    def callback(study, trial):
        value = "-" if trial.value is None else f"{trial.value:.5f}"
        try:
            best = f"{study.best_value:.5f}"
        except ValueError:
            best = "-"
        epochs = f", {trial.user_attrs['epochs']} epochs" if "epochs" in trial.user_attrs else ""
        print(f"[{kind}] trial {trial.number:02d} {trial.state.name:8s} loss={value} "
              f"({trial.user_attrs.get('seconds', 0):.0f}s{epochs}) best={best}")
        recent = study.trials[-3:]
        if len(recent) == 3 and all(t.state == optuna.trial.TrialState.FAIL for t in recent):
            print(f"[{kind}] three consecutive failed trials: stopping search")
            study.stop()
    return callback

def run_search(kind, fold, device):
    study = optuna.create_study(direction="minimize", study_name=kind,
                                sampler=optuna.samplers.TPESampler(seed=SEED, n_startup_trials=N_STARTUP[kind]),
                                pruner=PRUNER_FACTORY[kind]())
    study.enqueue_trial(DEFAULT_SUGGESTED[kind])
    durations = []
    n_trials = N_TRIALS_SEARCH[kind]
    print(f"[{kind}] Running search for {n_trials} trials...")
    study.optimize(make_objective(kind, fold, device, durations),
                   n_trials=n_trials, callbacks=[make_callback(kind)],
                   gc_after_trial=True, catch=(Exception,))
    return study
```

```
def confirm_candidates(kind, study, fold, device):
    complete = sorted([t for t in study.trials if t.state == COMPLETE], key=lambda t: t.value)
    if not complete:
        return []
    default = next((t for t in complete if t.number == 0), None)
    ordered = []
    for trial in [complete[0], default, *complete[1:K_CONFIRM]]:
        if trial is not None and all(trial.number != other.number for other in ordered):
            ordered.append(trial)
    rows = []
    for trial in ordered:
        row = {"trial": trial.number, "is_default": trial.number == 0, "params": dict(trial.params),
               "config": build_config(kind, trial.params), "score_search": float(trial.value),
               "score_confirm": None, "note": ""}
        try:
            result = run_guarded(EVALUATE[kind], row["config"], fold, device, None)
            row["score_confirm"] = float(result["val_loss"])
        except optuna.TrialPruned:
            row["note"] = "trial pruned"
        except Exception:
            row["note"] = "error: " + traceback.format_exc().strip().splitlines()[-1]
        confirm = "-" if row["score_confirm"] is None else f"{row['score_confirm']:.5f}"
        print(f"[{kind}] confirmation trial {row['trial']:02d}{' (default)' if row['is_default'] else ''}: "
              f"search fold {row['score_search']:.5f} | confirmation fold {confirm} {row['note']}")
        rows.append(row)
    return rows

def decide(kind, rows):
    decision = {"adopted": False, "config": dict(DEFAULT_CONFIG[kind]), "reason": "", "default_search": None,
                "default_confirm": None, "best_trial": None, "best_search": None, "best_confirm": None,
                "mean_gain": None, "relative_gain": None}
    default = next((r for r in rows if r["is_default"]), None)
    if default is not None:
        decision.update(default_search=default["score_search"], default_confirm=default["score_confirm"])
    if default is None or default["score_confirm"] is None:
        decision["reason"] = "default was not evaluated on both folds, keeping default"
        return decision
    candidates = [r for r in rows if not r["is_default"] and r["score_confirm"] is not None]
    if not candidates:
        decision["reason"] = "no candidate confirmed on second fold, keeping default"
        return decision
    mean_score = lambda r: 0.5 * (r["score_search"] + r["score_confirm"])
    best = min(candidates, key=mean_score)
    gain = mean_score(default) - mean_score(best)
    on_both_folds = best["score_search"] < default["score_search"] and best["score_confirm"] < default["score_confirm"]
    decision.update(best_trial=best["trial"], best_search=best["score_search"], best_confirm=best["score_confirm"],
                    mean_gain=gain, relative_gain=gain / mean_score(default))
```

```
    if gain >= MIN_GAIN and on_both_folds:
        decision.update(adopted=True, config=best["config"],
                        reason=f"better than default on both folds, mean gain {gain:.5f} >= MIN_GAIN {MIN_GAIN}")
    else:
        reasons = []
        if not on_both_folds:
            reasons.append("not better than default on both folds")
        if gain < MIN_GAIN:
            reasons.append(f"mean gain {gain:+.5f} < MIN_GAIN {MIN_GAIN}")
        decision["reason"] = "keeping default: " + "; ".join(reasons)
    return decision

def write_outputs(results, errors, started, complete):
    suffix = "_SMOKE_TEST" if SMOKE_TEST else ""
    final = {LABEL[kind]: (results[kind]["decision"]["config"] if kind in results else dict(DEFAULT_CONFIG[kind]))
             for kind in LABEL}
    report = {"complete": complete, "smoke_test": SMOKE_TEST, "elapsed_minutes": round((time.time() - started) / 60, 1),
              "protocol": {"search_fold": SEARCH_FOLD, "confirm_fold": CONFIRM_FOLD, "seed": SEED,
                           "min_gain": MIN_GAIN, "k_confirm": K_CONFIRM, "n_trials_search": N_TRIALS_SEARCH},
              "models": {LABEL[kind]: to_jsonable(value) for kind, value in results.items()},
              "errors": errors}
    with open(f"{OUTPUT_DIR}/best_hyperparameters{suffix}.json", "w") as f:
        json.dump(to_jsonable(final), f, indent=4)
    with open(f"{OUTPUT_DIR}/tuning_report{suffix}.json", "w") as f:
        json.dump(to_jsonable(report), f, indent=2)

def print_summary(results):
    rows = []
    for kind in LABEL:
        if kind not in results:
            rows.append({"model": LABEL[kind], "decision": "NOT TUNED"})
            continue
        d, counts = results[kind]["decision"], results[kind]["counts"]
        fmt = lambda v: "-" if v is None else f"{v:.5f}"
        rows.append({"model": LABEL[kind], "trials (complete/pruned/failed)":
                     f"{counts.get('COMPLETE', 0)}/{counts.get('PRUNED', 0)}/{counts.get('FAIL', 0)}",
                     "default (search/confirm)": f"{fmt(d['default_search'])} / {fmt(d['default_confirm'])}",
                     "best (search/confirm)": f"{fmt(d['best_search'])} / {fmt(d['best_confirm'])}",
                     "mean gain": "-" if d["mean_gain"] is None else f"{d['mean_gain']:+.5f} ({d['relative_gain']:+.1%})",
                     "adopted": "TUNED" if d["adopted"] else "default"})
    print(pd.DataFrame(rows).to_string(index=False))
    for kind in results:
        print(f"  {LABEL[kind]}: {results[kind]['decision']['reason']}")
```

In [8]:

```
# Step 8 - Sequential Execution Pipeline (Stable on GPU T4)
def run_tuning_pipeline():
    started = time.time()
    print("=== STARTING HYPERPARAMETER TUNING PIPELINE (ResNet, CatBoost, FT-Transformer) ===")
```

```python
# 1. Data Loading
X, Y, feature_cols, g_cols, c_cols, dummy_cols = load_and_preprocess_data()
print(f"Dataset Loaded -> Shape X: {X.shape}, Shape Y: {Y.shape}")

# 2. Pre-computing Fold Transformations
folds = prepare_cached_folds(X, Y, feature_cols, g_cols, c_cols, dummy_cols)
print(f"Inputs Ready in RAM after {(time.time() - started) / 60:.1f} min | "
      f"ResNet input: {folds[SEARCH_FOLD]['nn_train'].shape[1]} columns, "
      f"Raw input: {folds[SEARCH_FOLD]['raw_train'].shape[1]} columns")

results, errors = {}, {}
models_order = ["resnet", "catboost", "ft"]

# Sequential execution to prevent VRAM/CUDA conflicts in Jupyter
for kind in models_order:
    print(f"\n--- STARTING TUNING FOR {LABEL[kind].upper()} ---")
    try:
        # Stage 1: Search
        study = run_search(kind, folds[SEARCH_FOLD], DEVICE)
        counts = {}
        for trial in study.trials:
            counts[trial.state.name] = counts.get(trial.state.name, 0) + 1

        try:
            study.trials_dataframe().to_csv(f"{OUTPUT_DIR}/trials_{kind}{'_SMOKE_TEST' if SMOKE_TEST else ''}.csv", index=False)
        except Exception:
            pass

        if counts.get("COMPLETE", 0) < MIN_TRIALS_FOR_A_SEARCH:
            print(f"[{kind}] WARNING: only {counts.get('COMPLETE', 0)} completed trials (< {MIN_TRIALS_FOR_A_SEARCH})")

        # Stage 2: Confirmation
        rows = confirm_candidates(kind, study, folds[CONFIRM_FOLD], DEVICE)
        decision = decide(kind, rows)

        results[kind] = {
            "decision": decision, "counts": counts, "candidates": rows,
            "finished_after_min": round((time.time() - started) / 60, 1)
        }
        write_outputs(results, errors, started, complete=False)
        print(f"[{kind}] DECISION: {decision['reason']}")

    except Exception:
        errors[kind] = traceback.format_exc()
        print(f"[{kind}] ERROR, default value kept:\n{errors[kind]}")

    finally:
```

```
            gc.collect()
            if torch.cuda.is_available():
                torch.cuda.empty_cache()

    write_outputs(results, errors, started, complete=True)
    print(f"\n=== COMPLETED in {(time.time() - started) / 60:.1f} min ===")
    print_summary(results)
    return results
```

In [9]:

```
# Step 9 - Pipeline Execution
RESULTS = run_tuning_pipeline()
print()
suffix = "_SMOKE_TEST" if SMOKE_TEST else ""
print(open(f"{OUTPUT_DIR}/best_hyperparameters{suffix}.json").read())
```

```
=== STARTING HYPERPARAMETER TUNING PIPELINE (ResNet, CatBoost, FT-Transformer) ===
Dataset Loaded -> Shape X: (21948, 887), Shape Y: (21948, 206)
Pre-computing 5 stratified folds...
Inputs Ready in RAM after 0.4 min | ResNet input: 952 columns, Raw input: 887 columns

--- STARTING TUNING FOR RESNET ---
[resnet] Running search for 25 trials...
[resnet] trial 00 COMPLETE loss=0.01689 (12s, 17 epochs) best=0.01689
[resnet] trial 01 COMPLETE loss=0.01640 (10s, 35 epochs) best=0.01640
[resnet] trial 02 COMPLETE loss=0.01714 (4s, 23 epochs) best=0.01640
[resnet] trial 03 COMPLETE loss=0.01688 (5s, 16 epochs) best=0.01640
[resnet] trial 04 COMPLETE loss=0.01658 (10s, 35 epochs) best=0.01640
[resnet] trial 05 COMPLETE loss=0.01712 (5s, 16 epochs) best=0.01640
[resnet] trial 06 COMPLETE loss=0.01686 (5s, 16 epochs) best=0.01640
[resnet] trial 07 COMPLETE loss=0.01640 (5s, 35 epochs) best=0.01640
[resnet] trial 08 COMPLETE loss=0.01681 (8s, 15 epochs) best=0.01640
[resnet] trial 09 PRUNED   loss=0.02045 (0s) best=0.01640
[resnet] trial 10 PRUNED   loss=0.01824 (0s) best=0.01640
[resnet] trial 11 PRUNED   loss=0.01739 (0s) best=0.01640
[resnet] trial 12 PRUNED   loss=0.01978 (0s) best=0.01640
[resnet] trial 13 COMPLETE loss=0.01623 (15s, 26 epochs) best=0.01623
[resnet] trial 14 PRUNED   loss=0.01748 (0s) best=0.01623
[resnet] trial 15 COMPLETE loss=0.01643 (12s, 21 epochs) best=0.01623
[resnet] trial 16 PRUNED   loss=0.01728 (0s) best=0.01623
[resnet] trial 17 PRUNED   loss=0.01798 (0s) best=0.01623
[resnet] trial 18 PRUNED   loss=0.01857 (0s) best=0.01623
[resnet] trial 19 PRUNED   loss=0.01760 (0s) best=0.01623
[resnet] trial 20 PRUNED   loss=0.01652 (0s) best=0.01623
[resnet] trial 21 PRUNED   loss=0.02011 (0s) best=0.01623
[resnet] trial 22 PRUNED   loss=0.01986 (0s) best=0.01623
[resnet] trial 23 PRUNED   loss=0.01835 (0s) best=0.01623
[resnet] trial 24 COMPLETE loss=0.01631 (16s, 28 epochs) best=0.01623
[resnet] confirmation trial 13: search fold 0.01623 | confirmation fold 0.01652
[resnet] confirmation trial 00 (default): search fold 0.01689 | confirmation fold 0.01709
[resnet] confirmation trial 24: search fold 0.01631 | confirmation fold 0.01644
[resnet] confirmation trial 07: search fold 0.01640 | confirmation fold 0.01658
[resnet] DECISION: better than default on both folds, mean gain 0.00062 >= MIN_GAIN 5e-05

--- STARTING TUNING FOR CATBOOST ---
[catboost] Running search for 15 trials...
[catboost] trial 00 COMPLETE loss=0.01707 (339s) best=0.01707
[catboost] trial 01 COMPLETE loss=0.01706 (338s) best=0.01706
[catboost] trial 02 COMPLETE loss=0.01868 (197s) best=0.01706
[catboost] trial 03 COMPLETE loss=0.01701 (655s) best=0.01701
[catboost] trial 04 COMPLETE loss=0.01710 (655s) best=0.01701
[catboost] trial 05 COMPLETE loss=0.01682 (206s) best=0.01682
[catboost] trial 06 COMPLETE loss=0.01701 (203s) best=0.01682
[catboost] trial 07 COMPLETE loss=0.01938 (661s) best=0.01682
```

```
[catboost] trial 08 COMPLETE loss=0.01766 (353s) best=0.01682
[catboost] trial 09 COMPLETE loss=0.01681 (204s) best=0.01681
[catboost] trial 10 COMPLETE loss=0.01916 (674s) best=0.01681
[catboost] trial 11 COMPLETE loss=0.01682 (204s) best=0.01681
[catboost] trial 12 COMPLETE loss=0.01693 (203s) best=0.01681
[catboost] trial 13 COMPLETE loss=0.01702 (204s) best=0.01681
[catboost] trial 14 COMPLETE loss=0.01692 (203s) best=0.01681
[catboost] confirmation trial 09: search fold 0.01681 | confirmation fold 0.01679
[catboost] confirmation trial 00 (default): search fold 0.01707 | confirmation fold 0.01707
[catboost] confirmation trial 05: search fold 0.01682 | confirmation fold 0.01681
[catboost] confirmation trial 11: search fold 0.01682 | confirmation fold 0.01683
[catboost] DECISION: better than default on both folds, mean gain 0.00027 >= MIN_GAIN 5e-05

--- STARTING TUNING FOR FTTRANSFORMER ---
[ft] Running search for 15 trials...
[ft] trial 00 COMPLETE loss=0.01684 (484s, 33 epochs) best=0.01684
[ft] trial 01 COMPLETE loss=0.01714 (512s, 35 epochs) best=0.01684
[ft] trial 02 COMPLETE loss=0.01791 (498s, 35 epochs) best=0.01684
[ft] trial 03 PRUNED   loss=0.02093 (0s) best=0.01684
[ft] trial 04 COMPLETE loss=0.01695 (513s, 35 epochs) best=0.01684
[ft] trial 05 PRUNED   loss=0.02063 (0s) best=0.01684
[ft] trial 06 COMPLETE loss=0.01689 (500s, 35 epochs) best=0.01684
[ft] trial 07 PRUNED   loss=0.01823 (0s) best=0.01684
[ft] trial 08 PRUNED   loss=0.02022 (0s) best=0.01684
[ft] trial 09 PRUNED   loss=0.01990 (0s) best=0.01684
[ft] trial 10 PRUNED   loss=0.01714 (0s) best=0.01684
[ft] trial 11 PRUNED   loss=0.01893 (0s) best=0.01684
[ft] trial 12 PRUNED   loss=0.01834 (0s) best=0.01684
[ft] trial 13 PRUNED   loss=0.01855 (0s) best=0.01684
[ft] trial 14 PRUNED   loss=0.01842 (0s) best=0.01684
[ft] WARNING: only 5 completed trials (< 8)
[ft] confirmation trial 00 (default): search fold 0.01684 | confirmation fold 0.01713
[ft] confirmation trial 06: search fold 0.01689 | confirmation fold 0.01709
[ft] confirmation trial 04: search fold 0.01695 | confirmation fold 0.01722
[ft] DECISION: keeping default: not better than default on both folds; mean gain +0.00000 < MIN_GAIN 5e-05

=== COMPLETED in 194.6 min ===
        model trials (complete/pruned/failed) default (search/confirm) best (search/confirm)        mean gain adopted
       ResNet                         12/13/0        0.01689 / 0.01709     0.01623 / 0.01652 +0.00062 (+3.7%)   TUNED
FTTransformer                          5/10/0        0.01684 / 0.01713     0.01689 / 0.01709 +0.00000 (+0.0%) default
     CatBoost                         15/0/0        0.01707 / 0.01707     0.01681 / 0.01679 +0.00027 (+1.6%)   TUNED
  ResNet: better than default on both folds, mean gain 0.00062 >= MIN_GAIN 5e-05
  CatBoost: better than default on both folds, mean gain 0.00027 >= MIN_GAIN 5e-05
  FTTransformer: keeping default: not better than default on both folds; mean gain +0.00000 < MIN_GAIN 5e-05

{
    "ResNet": {
        "lr": 0.0011822319558820668,
```

```
        "dropout": 0.42748834245090617,
        "weight_decay": 8.466460557534677e-06,
        "hidden_dim": 768,
        "batch_size": 128
    },
    "FTTransformer": {
        "lr": 0.001,
        "dropout": 0.2,
        "weight_decay": 1e-05,
        "d_token": 64,
        "n_blocks": 2,
        "attention_heads": 4,
        "batch_size": 64
    },
    "CatBoost": {
        "depth": 4,
        "l2_leaf_reg": 19.012973597638837,
        "learning_rate": 0.10338380617159558,
        "subsample": 0.8285511060504747,
        "iterations": 250
    }
}
```